Week 5 Session 2

Week 5, Session 2 — Build Sprint: Your First Agent with n8n or Voiceflow

IDSC 4101: Agentic AI for Business Applications

Session type: Hands-on build sprint + peer demo **Tools used:** n8n (workflow automation) or Voiceflow (conversational agent builder) **Assignment released:** Reflection Post 3 — due before Week 6 Session 1

SLIDE DECK

SLIDE 1 — Title Slide

Title: Build Sprint 2 **Subtitle:** Your First Working Agent — Design, Build, Demo **Course:** IDSC 4101: Agentic AI for Business Applications | Week 5, Session 2

Speaker Notes: Welcome back. You decided on your use case and your tool at the end of Session 1. Today you build. I want to be honest about what this session is: it's compressed, it's messy, and at some point your agent will do something you didn't expect. That's not a problem — that's the point. The goal of this build sprint isn't to produce a polished product. It's to give you direct experience with the gap between designing an agent and deploying one. Every concept from Weeks 4 and 5 — goal specification, tool connections, memory, guardrails, failure handling — will become concrete and personal in the next 75 minutes. Let's not waste any of it.

SLIDE 2 — Session Structure and Time Budget

Title: 75 Minutes — Here's How We Use Them

Time allocation:

Phase	Duration	What happens
Tool orientation	10 min	Instructor live demo: building the simplest possible agent in n8n and Voiceflow — one workflow each, 5 minutes each. Students observe only.
Build sprint	50 min	Teams build. Instructor circulates. No presentations, no interruptions.
Peer demo	10 min	Each team demos to one other team. Happy path + edge case.
Class debrief	5 min	What surprised you? What broke first? One observation per team.

The non-negotiable: Every team must have a working agent — even a minimal one — before the demo. “We ran out of time” is not an acceptable demo outcome. If you’re stuck, call me over immediately. Don’t wait.

Speaker Notes: The time allocation is tight by design. If you spend 20 minutes debating which use case to build, you’ll have 30 minutes to build and it won’t be enough. If your team hasn’t already decided on the use case and tool from Session 1 prep, do that in the next 2 minutes before the tool orientation starts. The tool orientation is 10 minutes total — I’ll show you just enough to start. The rest you’ll learn by doing. That’s intentional: the fastest way to understand n8n or Voiceflow is to have a problem you’re trying to solve in front of you while you’re using it, not to watch someone else solve a different problem.

SLIDE 3 — Tool Orientation: n8n in 5 Minutes

Title: n8n — The Minimum You Need to Start Building

Core concepts (shown live by instructor):

Workflow canvas: A visual flowchart where each box is a “node.” Data flows left to right (or top to bottom). Each node does one thing: trigger, transform, call an API, make a decision, send a message.

The three nodes you’ll use most: 1. **Trigger node** — starts the workflow. Options: manual trigger (click a button), webhook (HTTP request), schedule (every morning at 9am), or another app event (new row in Google Sheets). 2. **AI Agent node** — the LLM call. You define: which model, what system prompt (the agent’s instructions and constraints), what tools it has access to. 3. **Output node** — what the workflow does with the result: send an email, write to a spreadsheet, post to Slack, return a response via webhook.

Live demo flow (instructor builds this in 5 minutes): - Trigger: manual (click “Test workflow”) - AI Agent: GPT-4o, system prompt = “You are a customer service assistant for a coffee brand called Roast & Co. Answer questions about our products. If you don’t know the answer, say so and offer to connect the customer with a human.” - Output: return the response as text

Speaker Notes: This is genuinely the simplest possible n8n agent. It has one loop cycle: receive input → LLM call → output. No memory beyond the current message, no tool use beyond the LLM itself, no error branching yet. That’s fine — it’s a starting point. Teams building in n8n will add complexity from here: connecting to a knowledge base, adding a conditional branch for escalation, logging results. But the scaffolding is on screen. Every team using n8n should be able to replicate this in about 3 minutes once I hand over to the build sprint. From there, your complexity is up to you.

SLIDE 4 — Tool Orientation: Voiceflow in 5 Minutes

Title: Voiceflow — The Minimum You Need to Start Building

Core concepts (shown live by instructor):

Canvas: A visual conversation flow. Each block represents a conversational step. Arrows show the path the conversation can take. Branching = the agent handles different user responses differently.

The four block types you’ll use most: 1. **Start block** — where every conversation begins. Add a welcome message here. 2. **AI Response block** — the LLM call. You define: the system prompt (what the agent knows and is allowed to do), the knowledge sources (documents or URLs the agent can reference), and the response style. 3. **Condition block** — branching logic. “If the user mentioned a complaint → go to escalation path. Otherwise → continue FAQ path.” 4. **End / Escalation block** — where the conversation terminates or routes to a human. Always include at least one.

Live demo flow (instructor builds this in 5 minutes): - Start block: “Hi! I’m the Roast & Co. assistant. What can I help you with today?” - AI Response block: system prompt = “You are a customer service assistant for Roast & Co. coffee. Answer questions about our products, brewing tips, and orders. If asked about returns or complaints, respond with empathy and route to a human.” - Condition block: if user intent = complaint → escalation path - End block: “Connecting you with our team now. Someone will be in touch within 2 hours.”

Speaker Notes: Same agent as the n8n demo, different tool. What you’ll notice is that Voiceflow makes the conversation branching very visual — you can literally see the paths a conversation can take. That’s its strength. It’s also its main constraint: every path has to be explicitly drawn. If a user says something completely unexpected, the agent falls back to the AI Response block — which is good — but if you haven’t tested those fallback paths, they can produce inconsistent responses. The most important thing to build in Voiceflow is a robust fallback: every branch that can end in “I don’t know” should connect to either a helpful out-of-scope response or an escalation path. Never leave a branch that dead-ends without a response.

SLIDE 5 — Build Requirements

Title: What Your Agent Must Do

Minimum requirements (every team):

☐ **Receive an input** — a user message, a trigger event, or a data prompt. Something starts the agent.

☐ **Apply at least one AI reasoning step** — the LLM call is used somewhere. The agent isn’t just a rule-based chatbot or a simple lookup.

☐ **Produce a useful output** — a response, an action, a routed result, a logged entry. Something happens at the end.

☐ **Handle one edge case explicitly** — the named failure from your Session 1 prep. Build a response for the input your agent doesn’t know how to handle.

Stretch goals (if you finish early):

→ Connect to a knowledge base (a Google Doc, a URL, or a PDF) so the agent answers from actual content, not just its training data

→ Add an escalation path with context handoff — the agent routes to a human AND passes along a summary of the conversation

→ Add logging — the agent writes each interaction to a Google Sheet or sends a Slack notification when it escalates

Speaker Notes: The minimum requirements are genuinely achievable in 50 minutes if you start building immediately. The stretch goals are for teams that move fast — and they map directly to what real enterprise deployments add on top of the basic agent. The knowledge base connection is the most valuable stretch goal: it’s the difference between an agent that answers from general AI training data and one that answers from your specific business content. If you finish the minimum and have time, build the knowledge base connection first.

SLIDE 6 — Guardrails in Practice: What to Build In

Title: Don’t Skip This — Guardrails Are Part of the Minimum Build

Three guardrails every team should implement:

1. Scope restriction in the system prompt Explicitly tell the agent what it is NOT allowed to do. This is written directly into the system prompt — not as a separate setting.

Example: "You are a customer service assistant for Roast & Co. You answer questions about our products, orders, and brewing tips. You do not discuss competitors. You do not provide legal or medical advice. You do not make promises about refunds or replacements — direct those requests to a human agent."

2. Explicit fallback response When the agent doesn't know the answer or the input is outside scope, it should say so clearly and not make something up.

Example: "If you don't know the answer or the question is outside your scope, say: 'That's outside what I can help with directly. Let me connect you with our team.' Do not guess or fabricate information."

3. Escalation trigger At least one condition that routes to a human. Define it explicitly, not vaguely.

Example: "If the user expresses frustration, mentions a refund, uses the words 'complaint,' 'broken,' or 'wrong order,' or asks to speak to a person — escalate immediately."

Speaker Notes: These three guardrails should take 5 to 10 minutes to implement. They're all system prompt text and a condition block — no additional tools required. I've seen dozens of student builds that work perfectly on the happy path and completely fail the first time someone types something unexpected. The fallback response alone prevents most of those failures. Before you go into demo mode, test your agent with an adversarial input — something it wasn't designed for. Does it handle it gracefully? If not, fix the fallback before the demo.

SLIDE 7 — Build Sprint (Displayed Throughout the 50-Minute Build)

Title: Build Sprint — In Progress

Displayed on screen for the full 50-minute build:

Your team's checklist:

☐ Trigger / entry point working ☐ LLM call configured (model selected, system prompt written) ☐ Scope restriction in system prompt ☐ Fallback response for out-of-scope inputs ☐ Escalation trigger defined and connected ☐ Happy path tested end-to-end ☐ Edge case tested and handled

If you're stuck: - System prompt not working → simplify it. One sentence at a time. - Tool connection failing → check credentials and API key. Ask the instructor. - Agent giving wrong answers → make the system prompt more specific. Add examples. - Running out of time → cut stretch goals. Build the minimum. Demo the minimum.

Call the instructor over if: you've been stuck for more than 5 minutes on the same problem.

Speaker Notes: Leave this slide on screen the entire build sprint. It serves as a constant reference and pacing guide. Circulate continuously — don't sit at the front. Your role in the build sprint is troubleshooter and quality reviewer, not lecturer. See the full facilitation notes in the Activity Plan section below for specific prompts and intervention strategies by problem type.

SLIDE 8 — Peer Demo Instructions

Title: Peer Demo — 10 Minutes

Demo structure (each team, 5 minutes total):

Part 1: The happy path (2–3 minutes) - Show the agent working as designed - Narrate: "This is what our agent does. Here's the input. Here's the reasoning step. Here's the output." - Demonstrate at least 2–3 different inputs that the agent handles well

Part 2: The edge case (1–2 minutes) - Show the input your agent was not designed to handle - Show how it responds (ideally: the fallback or escalation you built) - Narrate: "This is the guardrail we put in. Here's why we chose this response."

What the watching team does: - Note one thing the agent handles well - Note one thing they'd want to test if they had more time - Give feedback in 60 seconds after the demo

Speaker Notes: Assign demo pairs before the build sprint starts — or during the last 5 minutes of build time. Don't let teams self-select; mix use cases if possible (a Voiceflow team demos to an n8n team and vice versa). The cross-tool demos are more interesting — teams learn something about the other tool without having used it. Keep the peer feedback tight: one thing that works, one thing to test. You don't have time for full critique sessions. The real reflection is in the Reflection Post, not the live demo debrief.

SLIDE 9 — Class Debrief (5 minutes)

Title: What Just Happened?

Three questions, one answer each, one team each:

1. **"What was the first thing that broke?"** *(Call on the team that seemed most stuck during the build)*
2. **"What did you change in your system prompt between your first test and your demo?"** *(Call on a team that was iterating visibly — prompts on screen, multiple test runs)*
3. **"What would you build next if you had another 75 minutes?"** *(Call on the team that finished fastest and hit stretch goals)*

Speaker Notes: Five minutes is not enough for a full debrief — save the depth for the Reflection Post. These three questions are designed to surface the three most important learning moments: the failure, the iteration, and the aspiration. The first question normalizes failure — something broke for everyone, and that's the point. The second question surfaces the prompt engineering reality: good system prompts are written through iteration, not in one sitting. The third question connects the build sprint to the broader course arc — what you built today is the foundation for more complex deployments in Week 6 and the capstone.

SLIDE 10 — Reflection Post 3

Title: Reflection Post 3 — Due Before Week 6 Session 1

Prompt: 200–300 words. Answer all three questions:

1. **What does your agent do?** One clear, specific sentence. Not "it helps customers" — "it answers product questions for a fictional coffee brand, escalates complaints, and handles out-of-scope requests by connecting users to a human agent."
2. **What did you decide your agent should NOT be allowed to do autonomously — and why?** Describe one explicit constraint you encoded in the system prompt or as a condition. Explain the reasoning: what failure mode were you preventing?

3. **What was the hardest design decision your team made?** This could be a technical decision (which tool, which trigger), a business decision (where to put the escalation threshold), or a goal clarity decision (how specific to make the system prompt). Describe the decision and how you resolved it.

Grading note: This post is not a product description. It is a reflection on design thinking. A post that says "our agent worked great and we're proud of it" will not score well. A post that describes a constraint you almost didn't add, the failure it prevented, and what you'd do differently will score very well.

Speaker Notes: Read this slide aloud before dismissing the class. Students who don't understand the difference between a product description and a design reflection will write the wrong thing. Emphasize question 2 specifically — the "should NOT" question is the direct payoff from Week 4's failure mode analysis. The best posts will trace a line from the failure mode the team identified in Week 4, to the guardrail they designed in Session 1's blueprint, to the specific system prompt constraint they encoded in Session 2's build. That through-line is what you're looking for.

SLIDE 11 — What Comes Next

Title: Week 6 — AI in Products and Business Models

Where we're going: - Week 6 Session 1: AI-native products and business models — from individual agents to company strategy - Week 6 Session 2: Responsible AI in practice — bias, hallucination, security, regulatory landscape - Week 7: Strategy synthesis, capstone presentations, where is this going

Your build sprint connects to capstone: The agent you built today is a proof of concept. In the capstone, you'll design an AI solution for a real business sector. The thinking you did — goal specification, guardrails, edge cases — is the methodology you'll apply at business-problem scale.

Speaker Notes: Keep this brief — end on energy, not on lecture. The class has just spent 75 minutes building. They're tired and accomplished in equal measure. The Week 6 preview is a one-minute orienting note, not a full preview. The important message is: what you did today is not a one-off exercise. It's a repeatable methodology. You'll use it again.

SLIDE 12 — Close

Title: You Built an Agent.

What happened today: - You translated a use case into a working system - You wrote a system prompt that constrained agent behavior - You designed for failure — explicitly, before it happened - You demoed to peers and received feedback - You have direct experience with the gap between design and deployment

The sentence I want you to be able to say: *"I've built a working AI agent. I understand what it can and can't do, and I know why."*

That sentence, backed by today's build, is true. Use it.

Speaker Notes: This close matters. Students who've spent 75 minutes wrestling with an agent that wouldn't behave can leave feeling frustrated rather than accomplished. Reframe explicitly: the friction was the learning. Every moment where the agent did something unexpected — that was the course working. Every prompt revision — that was real AI product development at learning speed. They have direct experience that most business professionals don't. That's worth naming.

ACTIVITY PLAN

Activity: Build Sprint 2 — "Your First Agent" (50-minute build + 10-minute peer demo + 5-minute debrief)

Purpose: Translate the agent blueprint and failure mode analysis from Week 4 into a working deployed agent using n8n or Voiceflow. Students experience the gap between design and execution, practice prompt engineering through iteration, and build direct confidence with agent-building tools.

Team configuration: Same teams as Weeks 4–5 Session 1. Assign demo pairs before or at the start of the build sprint.

Phase 1: Tool orientation (10 minutes — instructor-led, students observe)

Instructor builds two minimal agents live: - n8n: manual trigger → AI Agent node (GPT-4o, simple system prompt) → text output. Time: 5 minutes. - Voiceflow: start block → AI Response block (system prompt) → condition block → escalation end block. Time: 5 minutes.

Goal: every student sees both tools produce a working result before they build. Do not go deep on features. Show only what's on Slide 3 and Slide 4. Answer clarifying questions only if they block a team from starting — defer feature questions to the build sprint.

Phase 2: Build sprint (50 minutes — teams build, instructor circulates)

Opening instruction (30 seconds): "Your checklist is on the screen. Build in order: trigger first, then the LLM call, then the output, then the guardrails, then test. Don't skip to features before the basics work. Call me over if you're stuck for more than 5 minutes."

Circulation strategy: Visit each team at least twice during the build. First pass (minutes 5–15): check that every team has a trigger and an LLM call configured. Second pass (minutes 25–35): check that every team has tested the happy path at least once and has started on the edge case. Third pass (minutes 40–50): check that every team has a complete demo scenario prepared.

Intervention guide by problem type:

Problem: System prompt producing wrong or inconsistent outputs First ask: "What does the system prompt say?" Then: "What output did you get?" Then: "What's the gap?" Most students write system prompts that are too general. Prompt them: "Add one more constraint. Be specific about what it should NOT say. Test again." One targeted constraint almost always improves the output significantly. Teach them to iterate in small steps, not rewrite the whole prompt.

Problem: Tool connection failing (API key, authentication) This is a blocking problem — don't leave a team stuck on credentials. Help them resolve it immediately or switch them to a simpler configuration that doesn't require the failing connection (e.g., remove the external knowledge base and use a rich system prompt instead).

Problem: Agent giving confident wrong answers (hallucination) This is a teachable moment, not a failure to fix. Ask: "What guardrail would prevent this?" They'll usually identify the fallback response pattern: "If you don't know, say so." Help them add it to the system prompt. Then test again. The before/after is a good demo moment.

Problem: Team is over-scoping and running out of time If a team is building a 10-step workflow with multiple integrations and they have 20 minutes left with nothing working end-to-end, intervene: "What's the minimum version that completes the checklist? Build that first. Stretch goals only if you have a working minimum." Be direct. Don't let teams heroically sprint toward a complex build that fails the demo.

Problem: Team is done early (before 40 minutes) Direct them to stretch goals in order: knowledge base connection first, then escalation with context handoff, then logging. If they've completed stretch goals and still have time, ask: "What's the adversarial input you haven't tested yet? Try 3 things that should break it." That final adversarial testing is always productive and generates good Reflection Post material.

Phase 3: Peer demo (10 minutes)

Each team demos to their assigned pair. 5 minutes per team: 2–3 minutes on the happy path, 1–2 minutes on the edge case. Watching team gives 60 seconds of feedback: one thing that works, one thing to test.

Instructor observation: circulate during demos. Listen for teams that skip the edge case demo — intervene and ask them to show it even if it doesn't work perfectly. "Show us what happens when you give it something it wasn't designed for" is a more honest and interesting demo than a polished happy path only.

Phase 4: Class debrief (5 minutes)

Three questions, one per team, as described in Slide 9 notes. Keep it fast. The goal is shared reflection, not full critique. End with the close from Slide 12.

Materials and setup required: - n8n: instructor account pre-configured with at least one working OpenAI/ Gemini API key. Students use their own accounts (free tier is sufficient) or a shared class credential provided via LMS. - Voiceflow: instructor account pre-configured. Students create free accounts before Session 2 (remind via LMS after Session 1). - Display: instructor's screen shared throughout tool orientation and available for student troubleshooting reference. - Reflection Post 3 prompt posted to LMS before Session 2 ends.

Assessment connection: Reflection Post 3 is the individual accountability artifact for this build sprint. The build itself is assessed as part of the Build Sprints and In-Class Activities grade (25% of course grade). Grading criteria: did the agent meet the minimum requirements? Was the edge case handled? Was the demo coherent? Completion is more important than sophistication — a simple agent that works is worth more than a complex one that doesn't demo.